

Secure Cloud Storage

This project's aim is to provide a secure cloud storage solution that ensures data privacy and integrity. We are building and securing a cloud object storage environment using tools like MinIO, Docker, and OpenSSL. The project will focus on implementing encrypted storage, access control, and secure data transfer protocols.

Defined Tools and Technologies

- **MinIO:** An open-source object storage server that is compatible with Amazon S3 APIs. It will be used to store and manage data securely.
 - **Docker:** A platform for developing, shipping, and running applications in containers. It will be used to deploy the MinIO server and other components of the storage solution.
 - **OpenSSL:** A robust toolkit for implementing SSL and TLS protocols. It will be used to encrypt data in transit and ensure secure communication between clients and the storage server.
-

Phase 1: Setting Up the Environment

In this first step, we will set up the environment required for our secure cloud storage solution. This includes installing Docker and Docker Compose, generating SSL/TLS certificates, and creating a `docker-compose.yml` file to spin up a standalone MinIO instance.

1. Generating the SSL/TLS Certificates

```
mkdir -p minio-env/certs minio-env/data
cd minio-env
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout certs/private.key \
  -out certs/public.crt \
  -subj "/CN=localhost"
```

- The above command generates a self-signed SSL/TLS certificate and a private key. The certificate is valid for 365 days and is stored in the `certs` directory.
- Why we do this: Generating SSL/TLS certificates is crucial for encrypting data in transit. It ensures that any data sent between the client and the server is secure and cannot be intercepted by malicious actors.

2. Creating the Docker Compose File

```
version: '3.8'

services:
  minio:
    image: quay.io/minio/minio:latest
```

```
container_name: minio-secure
ports:
  - "9000:9000"
  - "9001:9001"
environment:
  - MINIO_ROOT_USER=admin4030
  - MINIO_ROOT_PASSWORD=SuperSecurePassword123!
volumes:
  - ./data:/data
  - ./certs:/root/.minio/certs
command: server /data --console-address ":9001"
restart: unless-stopped
```

We do this because:

- Docker Compose allows us to define and manage multi-container applications. By using a `docker-compose.yml` file, we can easily set up and configure the MinIO server along with any other necessary services in a single command. This simplifies deployment and ensures that all components are correctly configured to work together.
- Certificate Mounting: The `docker-compose.yml` file will include volume mounts for the SSL/TLS certificates. This ensures that the MinIO server can access the necessary certificates for secure communication. MinIO automatically looks in `~/.minio/certs` for certificates on startup. Mounting our local directory there forces MinIO to boot in HTTPS mode
- Port separation: We explicitly define port 9000 for S3 API traffic and 9001 for the Web Management Console. This allows us to apply different network monitoring or firewall rules to API vs. human traffic later.
- Starting the environment: Once the `docker-compose.yml` file is created, we can start the MinIO server and any other services defined in the file using the following command:

```
docker-compose up -d

# check the status of the containers
docker ps

# check the logs of the containers
docker logs minio-secure
```

```
(secure-cloud-storage) ch4mp@fedora:~/OneDrive/Desktop/Data Science Projects/Secure-Cloud-Storage$ docker logs minio-secure
INFO: Formatting 1st pool, 1 set(s), 1 drives per set.
INFO: WARNING: Host local has more than 0 drives of set. A host failure will result in data becoming unavailable. ...

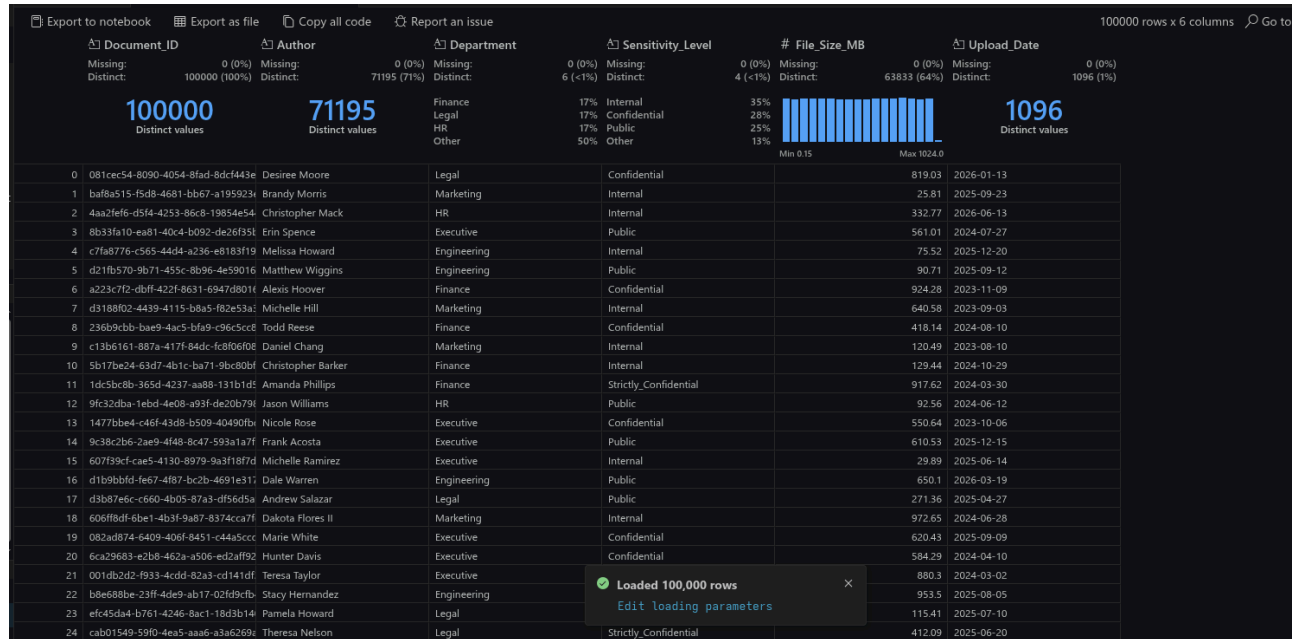
API: https://172.21.0.2:9000 https://127.0.0.1:9000
WebUI: https://172.21.0.2:9001 https://127.0.0.1:9001

Docs: https://docs.min.io
(secure-cloud-storage) ch4mp@fedora:~/OneDrive/Desktop/Data Science Projects/Secure-Cloud-Storage$
```

You should see output confirming MinIO is listening on `https://127.0.0.1:9000` and `https://127.0.0.1:9001`. If it says `http://`, the certificates were not mounted correctly.

Phase 2: Data Generation

- We will use the `data_generator.py` script to create sample data for testing our secure cloud storage solution. The script generates random user data and saves it in a CSV file format. This data will be used to simulate real-world scenarios and test the functionality of our storage system.



Phase 3: Security Controls (RBAC & Encryption)

For this phase, we will focus on implementing security controls to ensure that our cloud storage solution is secure and compliant with best practices. This includes setting up Role-Based Access Control (RBAC) and enabling Server-Side Encryption (SSE) for our storage buckets.

1. Install MinIO Client (mc) to manage the MinIO server and configure RBAC policies.

```
curl https://dl.min.io/client/mc/release/linux-amd64/mc \
  --create-dirs -o $HOME/minio-binaries/mc
chmod +x $HOME/minio-binaries/mc
export PATH=$PATH:$HOME/minio-binaries/

mc alias set localminio https://127.0.0.1:9000 admin4030
SuperSecurePassword123! --insecure
```

2. Create a new bucket and enable server-side encryption (SSE) using a KMS key.

```
./mc mb localminio/hr-data --insecure
./mc mb localminio/finance-data --insecure

./mc admin user add localminio HR_User HRPwd123! --insecure
./mc admin user add localminio Finance_User FinPwd123! --insecure
```

```
ch4mp@fedora:~/minio-binaries$ ./mc mb localminio/hr-data --insecure
Bucket created successfully `localminio/hr-data`.
ch4mp@fedora:~/minio-binaries$ ./mc mb localminio/finance-data --insecure
Bucket created successfully `localminio/finance-data`.
ch4mp@fedora:~/minio-binaries$ ./mc admin user add localminio HR_User HRPwd123! --insecure
Added user `HR_User` successfully.
ch4mp@fedora:~/minio-binaries$ ./mc admin user add localminio Finance_User FinPwd123! --insecure
Added user `Finance_User` successfully.
ch4mp@fedora:~/minio-binaries$
```

- We use the `mc` command-line tool to create buckets and manage users. The `--insecure` flag is used because we are using self-signed certificates for our MinIO server.
- A bucket is a logical container for storing objects (files) in MinIO. By creating separate buckets for different departments (HR and Finance), we can enforce access control policies and ensure that only authorized users can access the data.
- KMS (Key Management Service) is used to manage encryption keys for server-side encryption. By enabling SSE with a KMS key, we ensure that the data stored in the buckets is encrypted at rest, providing an additional layer of security.

3. Set up RBAC policies to restrict access to the buckets based on user roles.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:*"],
    "Resource": ["arn:aws:s3:::hr-data/*", "arn:aws:s3:::hr-data"]
  }]
}
```

This JSON policy allows the `HR_User` to perform any action on the `hr-data` bucket and its contents. Similar policies can be created for the `Finance_User` to restrict access to the `finance-data` bucket. To apply these policies, we will use the `mc` command-line tool to set the policies for each user.

```
mc admin policy create localminio hr-access hr-policy.json --insecure
mc admin policy attach localminio hr-access --user HR_User --insecure

mc admin policy create localminio finance-access finance-policy.json --insecure
mc admin policy attach localminio finance-access --user Finance_User --insecure
```

```
ch4mp@fedora:~/minio-binaries$ ./mc admin policy attach localminio hr-access --user HR_User --insecure
Attached Policies: [hr-access]
To User: HR_User
```

```
ch4mp@fedora:~/minio-binaries$ ./mc admin policy attach localminio finance-access --user Finance_User --insecure
Attached Policies: [finance-access]
To User: Finance_User
```

4. Enabling Server-Side Encryption (SSE) for the buckets ensures that all data stored in the buckets is encrypted at rest. This provides an additional layer of security, protecting sensitive information from unauthorized access. By using a KMS key for encryption, we can manage and rotate encryption keys as needed, further enhancing the security of our cloud storage solution.

- Stop running containers and remove the volumes to clear the data and certificates:

```
docker-compose down
```

- Add the KMS key to the docker-compose.yml file and restart the containers:

```
environment:  
  - MINIO_ROOT_USER=admin4030  
  - MINIO_ROOT_PASSWORD=SuperSecurePassword123!  
  - MINIO_KMS_SECRET_KEY=my-minio-  
key:K7MDENG/bPxRfiCYzU+z7aQxYx1EXAMPLEKEY=
```

- Restart the containers:

```
docker-compose up -d
```

- Enforce automatic SSE on the buckets

```
./mc encrypt set sse-s3 localminio/hr-data --insecure  
./mc encrypt set sse-s3 localminio/finance-data --insecure
```

```
ch4mp@fedora:~/minio-binaries$ ./mc encrypt set sse-s3 localminio/hr-data --insecure  
Auto encryption configuration has been set successfully for localminio/hr-data  
ch4mp@fedora:~/minio-binaries$ ./mc encrypt set sse-s3 localminio/finance-data --insecure  
Auto encryption configuration has been set successfully for localminio/finance-data  
ch4mp@fedora:~/minio-binaries$
```

- Verify that the buckets are encrypted:

```
./mc encrypt info localminio/hr-data --insecure  
./mc encrypt info localminio/finance-data --insecure
```

```
ch4mp@fedora:~/minio-binaries$ ./mc encrypt info localminio/hr-data --insecure  
Auto encryption 'sse-s3' is enabled  
ch4mp@fedora:~/minio-binaries$ ./mc encrypt info localminio/finance-data --insecure  
Auto encryption 'sse-s3' is enabled  
ch4mp@fedora:~/minio-binaries$
```

In this phase, we will perform and document 6 security tests to validate the security controls implemented in our MinIO environment. Each test will include the objective, procedure, expected result, and actual result. Included tests will cover unauthorized access attempts, testing RBAC boundaries, and verifying encryption.

The objective is to verify that authorized users can access their respective buckets and perform allowed actions. The procedure involves logging in as HR_User and attempting to upload a file to the **hr-data** bucket. The expected result is that the upload is successful, and the actual result should confirm this.

```
(secure-cloud-storage) ch4mp@fedora:~/OneDrive/Desktop/Data Science Projects/Secure-Cloud-Storage$ SHOME/minio-binaries/mc cp document_metadata.csv hruser/hr-data/ --insecure
```

The progress bar shows:

Name	Size
document_metadata.csv	8.53 MiB / 8.53 MiB

A green progress bar indicates completion.

```
(secure-cloud-storage) ch4mp@fedora:~/OneDrive/Desktop/Data Science Projects/Secure-Cloud-Storage$
```

46.84 MiB/s 0s

```
(secure-cloud-storage) ch4mpedfodora:~/OneDrive/Desktop/Data Science Projects/Secure-Cloud-Storage$ $HOME/minio-binaries/mc ls hruser/finance-data/ --insecure
mc: <ERROR> Unable to list folder. Access Denied.
(secure-cloud-storage) ch4mpedfodora:~/OneDrive/Desktop/Data Science Projects/Secure-Cloud-Storage$
```

```
(secure-cloud-storage) ch4mp@fedora:~/OneDrive/Desktop/Data Science Projects/Secure-Cloud-Storage$ curl -k https://127.0.0.1:9000/hr-data/
<?xml version="1.0" encoding="UTF-8"?>
<Error><Code>AccessDenied</Code><Message>Access Denied.</Message><BucketName>hr-data</BucketName><Resource>/hr-data/</Resource><RequestId>18C224B8BA3CD0C</RequestId><HostId>dd9025bab4ad464b049177c95eb6ef374d3b3fd1af9251148b658df7ac2e3e8</HostId></Error> (secure-cloud-storage) ch4mp@fedora:~/OneDrive/Desktop/Data Science Projects/Secure-Cloud-Storage$
```

6 / 8

```
curl http://127.0.0.1:9000
```

```
(secure-cloud-storage) ch4mp@fedora:~/OneDrive/Desktop/Data Science Projects/Secure-Cloud-Storage$ curl http://127.0.0.1:9000
Client sent an HTTP request to an HTTPS server.
(secure-cloud-storage) ch4mp@fedora:~/OneDrive/Desktop/Data Science Projects/Secure-Cloud-Storage$
```

5. Test 5: Server-Side Encryption (SSE) Verification Test The objective is to ensure that all data stored in the buckets is encrypted at rest using server-side encryption (SSE). The procedure involves uploading a file to the **hr-data** bucket and then verifying that the file is encrypted. The expected result is that the file is encrypted, and the actual result should confirm this.

```
mc encrypt info localminio/finance-data --insecure
```

```
ch4mp@fedora:~/minio-binaries$ ./mc encrypt info localminio/finance-data --insecure
Auto encryption 'sse-s3' is enabled
ch4mp@fedora:~/minio-binaries$
```

6. Test 6: Audit Log Capture and Review Test The objective is to verify that all access and actions performed on the MinIO server are logged for auditing purposes. The procedure involves reviewing the MinIO server logs to ensure that all actions are recorded. The expected result is that all actions are logged, and the actual result should confirm this.

```
# Terminal 1: Start the trace
./mc admin trace localminio --insecure

# Terminal 2: Trigger the violation
./mc ls hruser/finance-data/ --insecure
```

- Triggered violation

```
ch4mp@fedora:~/minio-binaries$ ./mc ls hruser/finance-data/ --insecure
mc: <ERROR> Unable to list folder. Access Denied.
ch4mp@fedora:~/minio-binaries$
```

- Violation captured in the trace

```
ch4mp@fedora:~/minio-binaries$ ./mc admin trace localminio --insecure
ch4mp@fedora:~/minio-binaries$ ./mc encrypt info localminio/finance-data --insecure
Auto encryption 'sse-s3' is enabled
ch4mp@fedora:~/minio-binaries$ ./mc admin trace localminio --insecure
2026-07-14T14:49:11.538 [403 Forbidden] s3.GetBucketLocation 127.0.0.1:9000/finance-data/?location= 172.21.0.1 600µs ↓ 519.89µs ↑ 93 B ↓ 304 B
2026-07-14T14:49:11.541 [403 Forbidden] s3.ListObjectsV2 127.0.0.1:9000/finance-data/?delimiter=%2F&encoding-type=url&fetch-owner=true&list-type=2&prefix= 172.21.0.1 636µs ↓ 598.346µs ↑ 93 B ↓ 304 B
```

Phase 5: Documentation & Risk Assessment

In this final phase, we will document the entire project, including the setup, configuration, security controls, and testing procedures. Additionally, we will conduct a risk assessment to identify potential risks associated with the MinIO cloud storage environment and recommend controls to mitigate them.

Risk Description	Impact	Likelihood	Recommended Controls
Data Interception in Transit	High	Low	Enforce SSL/TLS certificates for secure communication between clients and the storage server[cite: 1].
Unauthorized Data Access (Lateral Movement)	High	Medium	Implement Role-Based Access Control (RBAC) policies to restrict access based on departmental roles[cite: 1].
Physical Data Theft / Disk Compromise	High	Low	Enable Server-Side Encryption (SSE) with a Key Management Service (KMS) to encrypt data at rest[cite: 1].
Unauthenticated Public Exposure	High	High	Disable anonymous access and require valid credentials for all MinIO server requests[cite: 1].
Administrative Interface Compromise	High	Medium	Separate Web Management Console traffic to a dedicated port (9001) and enforce strong credentials[cite: 1].